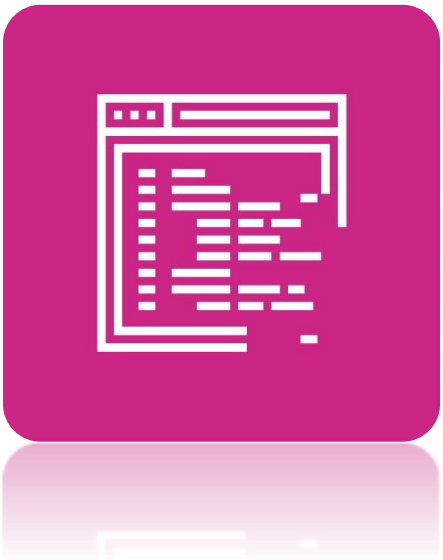


Short Guide
Webserver

1. REFERENCE ENVIRONMENT

This document refers to the following systems:

HW	New iPro models: **IPG115D Enhanced- Full** **IPG215D Enhanced- Full** **IPG315D Enhanced- Full** **IPG108D Enhanced** **IPG208D Enhanced** **IPL500D**
BIOS	**zupdater-2021071500 and subsequent**
TDB	**IPRO_20210715 and subsequent**
Isa WB	**ISaGRAF 6.6.8**

HW	New iPro models: **IPG115D Enhanced- Full** **IPG215D Enhanced- Full** **IPG315D Enhanced- Full** **IPG108D Enhanced** **IPG208D Enhanced** **IPL500D**
BIOS	**zupdater-2021071500 and subsequent**
TDB	**ARM_TARGET_L6_20210715 and subsequent**
ISaWB	**ISaGRAF 5.22 and ISaGRAF 6.5**

2. DESCRIPTION

The main purpose of the webserver is to allow the developer to create his own custom website that will be put inside the programmable controller. This custom website may be used for several purposes:

- Show the functionality of the unit\plant
- Show some specific functions for service operations
- Allow the end user to control the unit by remote through internet

This guide is not intended to explain the HTML language and JavaScript language functions but is made to give to the developer all the necessary instructions to integrate his own HTML code.

2.1 Requirements

To develop a custom website for the programmable controller, the developer should have as initial requirement a basic\good knowledge regarding the HTML programming language.
However, to use all the functionalities of the custom website, the following requirements are recommended:

- Basic\Good knowledge of HTML programming language
- Basic\Good knowledge of Java script programming language
- Basic\Good knowledge of CSS\Responsive programming

There is no need of a specific programming tool to develop the custom webpages. It's possible to program them directly with a text editor or to use an external program made for such operations.
Other than this it is recommended to have different browsers installed on the developing PC to check\debug the custom website with them.

2.2 Limitations

As limitations for the development of the custom webpages we have the following list:

- The custom website folder can have a maximum size of 4MB (including pages, images and scripts)
- The developer can use only client side languages (for example the PHP language cannot be used)

2.3 Website Structure

The custom website structure is strictly bound to the web pages' permissions and security. Mainly it is divided in "root" level and "folders" level.

- Root level: anything that is put into the "root" level will be always visible by anyone. The page index.html, which will be always the first page shown from the custom website, must be placed inside the "root" level. An item is considered at "root" level when it's not contained in a folder.

Folders's level: anything that is put into the "folders" level will be visible according to the permissions set in the control panel (see the section "permissions" regarding how to set all the permissions for the various folders inside the custom webpage. An item is considered at "folders" level when it is contained in a folder

A common structure used to setup the customer website is the following:

```
                    Index.html          (* main page in the root level *)
                    publicPage.html     (* public page in the root level *)
ImagesFolder/       (* folder that contains all the images used into the custom website. Folders level *)
ScriptFolder/       (* folder that contains all the scripts used into the custom website. Folders level *)
  User1WebPages/    (* folder that contains all the pages for the user 1 used into the custom website.
                                                                    Folders level *)
  User2WebPages/    (* folder that contains all the pages for the user 2 used into the custom website.
                                                                    Folders level *)
  User3WebPages/    (* folder that contains all the pages for the user 3 used into the custom website.
                                                                    Folders level *)
```

See the "Permissions" chapter for a detailed explanation of how the permissions work.

2.4 Header Imports

To be able to use the webserver functions, the developer needs to prepare some settings on the HTML header part. Those imports are needed to enable several specific functions, like the HTML commands or the JavaScript commands.

```
<head>
    <script language="javascript" type="text/javascript" src="/javascripts/jquery-1.4.3.js"></script>
    <script language="javascript" type="text/javascript" src="/javascripts/md5.min.js"></script>
    <script language="javascript" type="text/javascript" src="/javascripts/dixell.js"></script>
    <script language="javascript" type="text/javascript" src="/panel/css/j_utility.js"></script>
</head>
```

Text format:

```
<script language="javascript" type="text/javascript" src="/javascripts/jquery-1.4.3.js"></script>
<script language="javascript" type="text/javascript" src="/javascripts/md5.min.js"></script>
<script language="javascript" type="text/javascript" src="/javascripts/dixell.js"></script>
<script language="javascript" type="text/javascript" src="/panel/css/j_utility.js"></script>
```

IMPORTANT NOTE: these imports should be done in every page developed for the webserver. It's important also to keep the script order. Changing the script order may introduce errors in the webpage.
EXCEPTION: the script jquery-1.4.3.js is already present inside the iPro but it is possible for the developer to use a more recent version of this script. The developer may insert the script into another folder. This doesn't change the rule that this script should always be imported as first.

2.5 HTML read commands

There is a set of the available HTML instructions to read the ISaGRAF variables. The technique used to achieve this result, basically, is to use specific span classes to transform the span objects into something more useful.
With this kind of spans, it is possible to read the following types of variables: BOOL, DINT, STRING and REAL.

2.5.1 BOOL

It's possible to read a BOOL variable from the application. The BOOL variable itself can have only two available values (TRUE & FALSE) that usually need to be managed to show something more understandable for the final user like "On" or "Off" strings. This feature is already managed in the specific span code shown below:

Syntax:
Variable TrueString FalseString [Refresh]

Parameters:
Variable: the name of the variable in isagraf that needs to be shown (BOOL type)
TrueString: string parameter that will be shown in case of TRUE value on the selected variable
FalseString: string parameter that will be shown in case of FALSE value on the selected variable
Refresh (optional): refresh time (in seconds) between the readings

Errors:
"Var not found!" is shown if the variable name doesn't exist in the current application.
"Syntax error!" is shown if the parameters of the call are not consistent.
Examples:

1) Example of reading a BOOL variable (UnitStatus), showing an ON string in case of value=TRUE or OFF string in case of value FALSE. The variable value is updated every 10 seconds.

   UnitStatus "ON" "OFF" 10
2) Example of reading a BOOL variable (pbStatus), showing an Active string in case of value=TRUE or Inactive string in case of value FALSE. The variable is loaded only when the page is loaded.

   pbStatus "Active" "Inactive"

2.5.2 DINT

It's possible to read a DINT variable from the application. It is possible to use a DINT variable in 3 different ways: to show a pure integer value (isavar_dint), to show an integer value that will be multiplied by a gain value and that can have an offset value (isavar_decimal) or it is possible to perform a selection through a set of strings based on the variable value (isavar_desc).

Syntax:
Variable [Refresh]
Variable Divisor Offset DecimalNumbers [Refresh]
Variable Val0="String1" … Val99="String99" [Refresh]

Parameters:
Variable: the name of the variable in ISaGRAF that needs to be shown\used (DINT type)
Divisor: number used to divide the read variable (commonly it is 10 to obtain a decimal value)
Offset: number that will be added to the read variable
DecimalNumber: number of characters to be shown after the "," (in case of decimal value)
ValX: string that will be shown if the variable value is equal to X (up to 100 strings can be defined)
Refresh (optional): refresh time (in seconds) between the readings

Errors:
"Var not found!" is shown if the variable name doesn't exist in the current application.
"Syntax error!" is shown if the parameters of the call are not consistent.

Examples:

1) Example of reading a DINT variable (UnitStatus), showing the integer value on HTML. The variable value is updated every 10 seconds.

   UnitStatus 10

2) Example of reading a DINT variable (pbValue). The value is read in decimals, so a Divisor will be applied to the value and one digit after the "," will be shown. The variable is loaded only when the page is loaded.

   pbValue 10 0 1

3) Example of reading a DINT variable (pbStatus) in order to show a specific string. The variable value is updated every 10 seconds.

   pbStatus Val0="Working" Val1="Not working" Val2="Not configured"

2.5.3 STRING

It's possible to read a STRING variable from the application. The STRING variables have only one type of syntax available.

Syntax:
Variable [Refresh]

Parameters:
Variable: the name of the variable in ISaGRAF that needs to be shown (STRING type)
Refresh (optional): refresh time (in seconds) between the readings
Errors:
"Var not found!" is shown if the variable name doesn't exist in the current application.
"Syntax error!" is shown if the parameters of the call are not consistent.

Examples:

1) Example of reading a STRING variable (UnitConfiguration), showing the string value on HTML. The variable value is updated every 10 seconds.

   UnitConfiguration 10

2.5.4 REAL

It's possible to read a REAL variable from the application. The REAL variables have only one type of syntax available because a REAL number is supposed to have a decimal part.

Syntax:
 Variable Divisor Offset DecimalNumbers [Refresh]

Parameters:
Variable: the name of the variable in isagraf that needs to be shown\used (DINT type)
Divisor: number used to divide the read variable (commonly it's 10 to obtain a decimal value)
Offset: number that will be added to the read variable
DecimalNumber: number of characters to be shown after the "," (in case of decimal value)
Refresh (optional): refresh time (in seconds) between the readings

Errors:
"Var not found!" is shown if the variable name doesn't exist in the current application.
"Syntax error!" is shown if the parameters of the call are not consistent.

Examples:

1) Example of reading a REAL variable (PressureValue), showing the real value on HTML. The real will not need a divisor or an offset and will be shown with two decimals. The variable value is updated every 10 seconds.

   PressureValue 0 0 2 10

3. HTML WRITE commands

There is a set of the available HTML instructions to write the isagraf variables. The technique used to achieve this result, basically, is to use specific span classes in order to transform the span objects into something more useful. Please refer to the desired function to know how the result will look like.
With this kind of spans it is possible to write the following types of variables: BOOL, DINT, STRING and REAL.

3.1.1 BOOL

It's possible to write a BOOL variable in the application. The BOOL variable can be set in two different ways: as a toggle (basically the value set is the opposite of the current value) or to be forced to TRUE or FALSE value. For each case we have a different syntax. Note that when a BOOL variable is driven by a toggle function the value can be read back after a refresh time in order to keep the value shown and the value inside the application aligned.

Syntax:
Variable TrueString FalseString [Refresh]
Variable Value Text

Parameters:
Variable: the name of the variable in isagraf that needs to be written (BOOL type)
TrueString: string parameter that will be shown in case of TRUE value assigned to the selected variable
FalseString: string parameter that will be shown in case of FALSE value assigned to the selected variable
Refresh (optional): refresh time (in seconds) between the writings
Value: can be set as TRUE or FALSE to force a BOOL variable value
Text: text shown on the button created by the span

Representation:
Isamod_booltog: will be shown as a button with a string inside according to the value of the variable
Isamod_boolwrt: will be shown as a button with a string inside according to the text parameter
Errors:
"Var not found!" is shown if the variable name doesn't exist in the current application.
"Syntax error!" is shown if the parameters of the call are not consistent.

Examples:
1) Example of writing a BOOL variable (UnitStatus), showing an ON string in case of value=TRUE or OFF string in case of value FALSE. The variable value is updated with the toggle function and the value shown is updated every 10 seconds.

   UnitStatus "ON" "OFF" 10

2) Example of writing a BOOL variable (pbStatus) to true, showing "Enable" as button text.

   pbStatus TRUE "Enabled"

3.1.2 DINT

It's possible to write a DINT variable in the application. It is possible to set a DINT variable in 4 different ways: to force an integer value, to write an integer value, to write a decimal value or to perform a selection from a predefined list of strings. In the second and third cases the value can be read back after a refresh time in order to keep the value shown and the value inside the application aligned.

Syntax:
Variable Text [Refresh]
Variable Value Text

Variable Divisor Offset DecimalNumbers Text [Refresh]
Variable Val0="String1" … Val99="String99" Text [Refresh]

Parameters:
Variable: the name of the variable in isagraf that need to be shown\used (DINT type)
Text: text shown on the button created by the span
Divisor: number used to divide the variable during the reading or to multiply the variable during the writing
Offset: number that will be added to the read variable
DecimalNumber: number of characters to be shown after the "," (in case of decimal value)
ValX: string that will be shown to change the variable value in X (up to 100 strings can be defined)
Refresh (optional): refresh time (in seconds) between the writings

Representation:
Isamod_dint: will be shown as a text field with the value read and with a submit button on the side.
Isamod_dintwrt: will be shown as a button with a string inside according to the text parameter.
Isamod_decimal: will be shown as a text field with the value read and with a submit button on the side.
Isamod_desc: will be shown as a combo box with the selectable strings. The value in accordance with the string will be written into the application.

Errors:
"Var not found!" is shown if the variable name doesn't exist in the current application.
"Syntax error!" is shown if the parameters of the call are not consistent.

Examples:

1) Example of writing a DINT variable (UnitStatus), showing and modifying the integer value in a text field. The variable value is updated every 10 seconds.

   UnitStatus 10

2) Example of forcing a DINT variable (UnitStatus) with a constant value of 10. Only a button will be shown with the text passed as third parameter.

   UnitStatus 10 "Write 10"

3) Example of reading a DINT variable (SetValue). The value is read in decimals, so a Divisor will be applied to the value and one digit after the "," will be shown in the text field. When the value is written, instead, the opposite formula will be applied, subtracting the offset, multiplying the Divisor, and truncating the right decimals. In this case, the variable will be loaded only when the page is loaded.

   SetValue 10 0 1

Example of writing a DINT variable (pbStatus) selecting a specific string from a combo box. The variable value is updated every 10 seconds in order to keep updated the combo box selection.

pbStatus Val0="Working" Val1="Not working" Val2="Not configured"

3.1.3 STRING

It's possible to write a STRING variable in the application. The STRING variable has only one type of syntax available.

Syntax:
Variable Text [Refresh]

Parameters:
Variable: the name of the variable in isagraf that needs to be written (STRING type)
Text: text shown on the button created by the span
Refresh (optional): refresh time (in seconds) between the writings

Representation:
Isamod_string: will be shown as a text field with the value read and with a submit button on the side

Errors:
“Var not found!” is shown if the variable name doesn’t exist in the current application.
“Syntax error!” is shown if the parameters of the call are not consistent.

Examples:

1) Example on writing a STRING variable (UnitStatus), showing the current string value on the text field. On button click, the new value will be sent to the variable. The variable value is updated every 10 seconds.

    UnitStatus “Submit” 10

3.1.4 REAL

It’s possible to write a REAL variable in the application. The REAL variable has one type of syntax available.
Syntax:
 Variable Divisor Offset DecimalNumbers Text [Refresh]

Parameters:

Variable: the name of the variable in isagraf that needs to be shown\used (REAL type)
Divisor: number used to divide\multiply the variable
Offset: number that will be added when reading or subtracted when writing
DecimalNumber: number of characters to be shown after the “,” (in case of decimal value)
Text: text shown on the button created by the span
Refresh (optional): refresh time (in seconds) between the writings

Representation:
Isamod_real: will be shown as a text field with the value read and with a submit button on the side

Errors:
“Var not found!” is shown if the variable name doesn’t exist in the current application.
“Syntax error!” is shown if the parameters of the call are not consistent.

Examples:

1) Example of writing a REAL variable (PressureSet), showing the real value on the text field. The real will not need a divisor or an offset and will be shown with two decimals. The variable value will be updated on the button click and read back every 10 seconds.

    PressureSet 0 0 2 10

3.1.5 MULTIPLE WRITINGS

In HTML it is possible to use the tag <form> with the following command in order to perform a multiple variable writing. In any case, this kind of writing is slow, because it performs single consecutive writings on the variables (it is like writing the variables one by one).
In order to use a form and its button submit function, there are some rules to keep in mind when developing the page:

1) The page should contain only the form with the variables within. Any other kind of span object used to write or read variables may return unpredictable results
2) Only one isaform_submit must be present in the page
3) Only one of the isaform_* objects must be used for a single variable (no multiple writings on the same variable)

Syntax:
<form>
     Variable Parameters
     Variable Parameters

```
        <span class="isaform_*"> Variable Parameters </span>
        …
        <span class="isaform_submit"> Text </span>
</form>
```

Parameters:
Variable: the name of the variable in isagraf that needs to be written
Parameters: the parameters used from every single kind of isaform_* (see the "Available objects" list for an explanation of every single object type)
Text: text shown on the button created by the span to submit the writings
Available objects:
isaform_bool: it will look like a check box. The parameter will be the name of the variable.
isaform_dint: it will look like a text box. The parameter will be the name of the variable.
isaform_decimal: it will look like a text box. The parameters will be the same as isamod_decimal without the option "Refresh"
isaform_desc: it will look like a text box. The parameters will be the same as isamod_desc without the option "Refresh"
isaform_string: it will look like a text box. The parameters will be the same as isamod_string without the option "Refresh"
isaform_real: it will look like a text box. The parameters will be the same as isamod_real without the option "Refresh"

Errors:
"Var not found!" is shown if the variable name doesn't exist in the current application.
"Syntax error!" is shown if the parameters of the call are not consistent.

Examples:
Example of writing a form used to write several values at the same time. Unit status may be the On\Off status, the DelayOn can be the delay at the power on:

```
<form>
        <span class="isaform_bool"> UnitStatus </span>
        <span class="isaform_dint"> DelayOn </span>
        <span class="isaform_submit"> "Submit" </span>
</form>
```

4. HTML SHOW & HIDE commands

Without the use of Java script, it is possible to use two specific commands in order to hide or show certain parts of the custom website.
These commands will perform a condition check between an application variable and a constant value. If the comparison results in true, the portion of the website will be shown, otherwise it will be hidden.
It's possible to define two objects with this specific class: a span or a div with the title option. In these two cases, the standard use of the title option (tooltip) will not be available.
Syntax:
 …
 …
Parameters:
Variable: the name of the variable that will be used in the comparison
Operation: operation used to check the condition. The available actions are: =, <, >, <=, >=, <>.
Value: value of comparison for the operation
Refresh (optional): refresh time (in seconds) for updating the variable used in the comparison
Notes:
In order to have the isa_show object working, all the objects inside the span\div must be visible.
Errors:
"Var not found!" is shown if the variable name doesn't exist in the current application.
"Syntax error!" is shown if the parameters of the call are not consistent

4.1 JavaScript CGI for multiple variables reading

In case of reading a huge number of variables, it is possible to use a specific CGI to perform the operation.
In order to correctly use the CGI some knowledge on the JavaScript language is required.

With this CGI it is possible to read BOOL, DINT and STRING values.

4.1.1 CGI structure

The CGI structure to perform the call is the following:
http://Controller_IP/cgi-bin/xjgetvar.cgi?name=PARAMETERES
PARAMETERS is the list of the variables that we are going to read with the CGI to use their values in the custom website. The variable names must be declared in sequence using the pipeline character (|) as divisor. For example: SETPOINT|UNITSSTATUS|ERROR
Other than the divisor character, there are some rules to follow to create the PARAMETERS list:

- All the variables names must be in uppercase letters
- All the variables read from the CGI must have an address in the application
- No space character allowed between a variable name and the pipeline characters
- In order to read a full array, define only the array name
- In order to read an element of an array, define the array name followed by square brackets containing the index of the element that we want to read

4.1.2 CGI result

After performing the CGI call, the result given back will have a json structure and will be usable to populate the web page.
DINT variables will show their current values.
BOOL variables will show 0 in case of false or 1 in case of true
STRING variables will show their string value within the “”
Example:
http://Controller_IP/cgi-bin/xjgetvar.cgi?name=SETPOINT|BAND|UNITSSTATUS|ALARM

```
{“values”:
        [
                {“name”:”SETPOINT”, ”value”:[250]},
                {“name”:”BAND”, ”value”:[40]},
                {“name”:”UNITSTATUS”, ”value”:[“Unit on”]},
                {“name”:”ALARM”, ”value”:[0]},
        ]
}
```

4.1.3 CGI errors

The CGI can give back the following errors:

- Variable not found: this error appears if the variable name is misspelled, if the variable name doesn’t exist in the application or if it is not written in uppercase letters in the CGI call
- Array index out of bounds: this error it’s shown when reading a single element of an array and the index is higher than the number of elements of the array

4.1.4 CGI javascript function

As said before, in order to use the CGI, some knowledge on the JavaScript programming is needed. Anyway here we have a function to start using this feature. This function should be put into a js file that will be imported in every page that needs to use it:

```
var AJAX_CGI_URL = "/cgi-bin/xjgetvar.cgi";

function getAjaxData(AJAX_PARAMS)
{
        $.ajax({
                type: "GET",
                url: AJAX_CGI_URL,
                dataType : 'json',
                cache: false,
                data: { name: AJAX_PARAMS }
                })
                .done(function( obj ) {
                        updateData(obj);
                })
                .fail(function( obj ) {
```

```
                    })
                    .always(function( obj ) {
                    });
}

function getData(AJAX_PARAMS, TimeOut)
{
        getAjaxData(AJAX_PARAMS);
        window.setTimeout(function(){ getData(AJAX_PARAMS, TimeOut);}, TimeOut);
}

function getSingleData(AJAX_PARAMS)
{
        getAjaxData(AJAX_PARAMS);
}
```

With this function it is possible to perform a multiple reading of variables from any webpage just calling the functions getData (in case of recursive call to have the refresh) or the function getSingleData (in case of single shot reading of the variables).

Both these functions will give back an object named obj that will contain the result of the CGI call and that can be used to retrieve the values of the variables.

5. JavaScript CGI for files management

In case of log file managing, it is possible to use a specific CGI to perform the operation. To correctly use the CGI some knowledge on the JavaScript language is required.
With this CGI it is possible to read the log files with a specific parametrization to achieve the desired result.
The user who needs to use this CGI call, needs to have the permission to see the files tab of the control panel (see the permissions chapter for further information).

5.1.1 CGI structure

The basic CGI structure to perform the call is the following:
http://Controller_IP/cgi-bin/getfile.cgi?log=myFile&what=log&ext=Option1&disp=Option2
myFile: is the log file name selected for the operations
ext: optional parameter used to define the file extension. The default (no parameter declared) is .txt. The other available selections are:

- ext=xml
- ext=csv

Note: the log file text is created inside the application. In order to correctly manage an xml file or a csv file, the log file should be written as one of those kinds of files.
disp: optional parameter used to define how to show the log file. The available options are:

- inline=open the file and put the text into a javascript variable or as text content of an HTML element
- attachment=perform the download of the file
- app=open the file with the predefined app for that kind of file (example Excel for csv files)

Note: javascript does not allow the download of files. In order to use the attachment option, it is necessary to perform a direct call to the CGI.

5.1.2 CGI result

After performing the CGI call, the result given back will depend on the type of call performed. The developer will be able to decide if the file will be downloaded, if the file will be opened in the HTML context or if the file will be opened with a predefined application.

5.1.3 CGI errors

In case of any error while using the CGI, the result will be the download\opening of a void file named “nothing” without text inside.

5.1.4 CGI javascript function

As said before, in order to use the CGI, some knowledge on the javascript programming is needed.
Anyway here we have a small function to start using this feature. This function should be put into a .js file

that will be imported in every page that needs to use it:
1)

```
function getAjaxLog(AJAX_PARAMS)
{
	$.ajax({
		type: "GET",
		url: LOG_CGI_URL,
		dataType : 'json',
		cache: false,
		data: {
			log: AJAX_PARAMS,
			what: 'log',
			disp: 'inline'
		}
		})
		.done(function( objLog ) {
			updateLog(objLog);
		})
		.fail(function( objLog ) {
			updateLog(objLog);
		})
		.always(function( objLog ) {
			updateLog(objLog);
		});
}

function getLog(AJAX_PARAMS)
{
	getAjaxLog(AJAX_PARAMS);
}
```

With this function it is possible to read a log file text and put it into a javascript object named objLog. The text of the log file will be available in the responseText property and will be ready to be manipulated through javascript.

2)

```
function downloadLog(AJAX_PARAMS)
{
	window.location.href = ".." + LOG_CGI_URL + "?log=" + AJAX_PARAMS +
"&what=log&disp=attachment";
}
```

With this function it is possible to perform the download of a log file from the custom website.

6. JavaScript functions for users management

There are three functions available to manage different users with their own permissions to see the custom website.
Please refer to the permissions chapter for further information.

6.1.1 Login function

It's possible for the developer to perform a login on the custom website for one of the users defined in the control panel access.
To do this, a function to perform the login is available. This function manages both the login and the check if the user exists or if the password matches:
magcheck_login(user, pass).val()
Passing the user and his password as parameters, the function will give back a true .val() if the user is recognized and the password matches. In any other case the .val() will be equal to false.

6.1.2 Logout function

It is possible for the developer to develop a logout function on the custom website.
To do this, a function to perform the logout is available.
magcheck_logout()
After the logout, the pages available only with certain permissions will go back to the hidden\not available status.

6.1.3 User check function

It is possible for the developer to perform a user check on the custom website for the logged user.
To do this, is available a function to perform the user check.
magcheck_user()
The result of this function will give back the currently logged user or null if no user is logged in.

6.1.4 access_error.html

It's possible for the developer to create a specific error page in case of wrong login or missing login. This page should be put in the "root" level and will be shown when a user without permissions or with wrong permissions will try to see a protected page. If the page doesn't exist, a 401-error page will be shown

6.1.5 Javascript implementation

Below are some examples of how to implement the described functions:

```
function Login()
{
	if (magcheck_login(USER, PASS).val()) == true)
	{
		/* if the check returns true, the user and password match */
		/* here we put a redirect to the main page for that user after the login */
	}
	else
	{
		/* if the check returns false, the user and password miss match */
		/* here we put the code used in case of wrong login */
	}
}

function Logout ()
{
	/* logout for the current user */
	magcheck_logout();
	/* code to be performed after the logout */
}

function getUser()
{
	var activeUser = magcheck_user();
	/* code to be performed eventually for a specific user */
}
```

7. User permissions management

On the programmable controller it is possible to manage up to three different users other than the admin user.
The admin user can always see any part of the control panel and website. The admin password can be managed from the accounts tab in the control panel.
The custom users can have different permission levels. Both the username and the password for the custom users can be managed from the account tab in the control panel.

Refer to “Control panel” short guide for the full usage of the programmable controller Control Panel
Besides managing the users and the passwords, in the accounts tab of the control panel it is also possible to manage the permissions for the webpages.

Standard website (panel)				
	User1	User2	User3	Free access
Login names ->	[User1]	[User2]	[User3]	
Home	Always free			
Variables	☐	☐	☐	☐
Configure	☐	☐	☐	☐
Files	☐	☐	☐	☐
Account	☐	☐	☐	☐
Advanced	☐	☐	☐	☐
Firewall	☐	☐	☐	☐

Custom website				
	User1	User2	User3	Free access
[root]	Always free			
js	☐	☐	☐	☐
webpages	☐	☐	☐	☐
images	☐	☐	☐	☐
styles	☐	☐	☐	☐

OK Export...

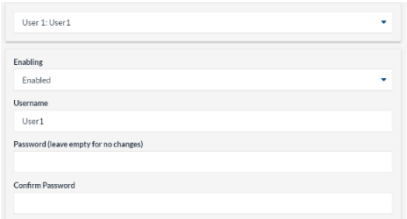
As the picture shows, there are two tables available on this page:

- Standard website: this table refers to the control panel always present inside the programmable controller. Checking the boxes under one of the users will allow it to be able to enter that specific page when logging into the control panel (enabling it on the menu). By setting the Free Access checkbox the developer will leave the page available to anyone (even not logged people).
- Custom website: following the levels rule described in the Website structure chapter under the custom website table it is possible to see a list of all the folders present inside the custom website. Also, in this case it is possible to restrict a folder to a single user view or to put it as free access in order to leave the folder available to anyone.

The base idea is to create a specific folder for every user defined, in order to keep the different pages split between the different users.
A common way to setup the custom website permissions is the following:

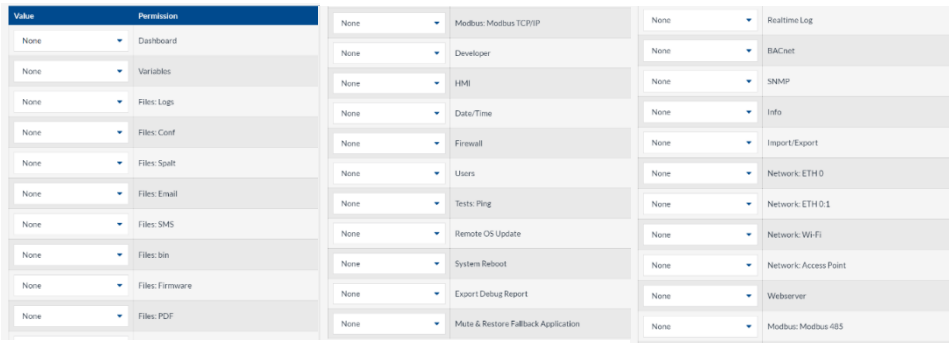
- Scripts folders are usually set as Free Access
- Images folders are usually set as Free Access
- Styles folders are usually set as Free Access
- User1 folders are usually set for User1, User2 and User3 (final user, service, OEM)
- User2 folders are usually set for User2 and User3 (service, OEM)
- User3 folders are usually set only for User3 (OEM)

Note that the developer is not obliged to declare all the Users. If a user is not needed, the developer must leave that permissions column without checks.
An example of common configuration is:



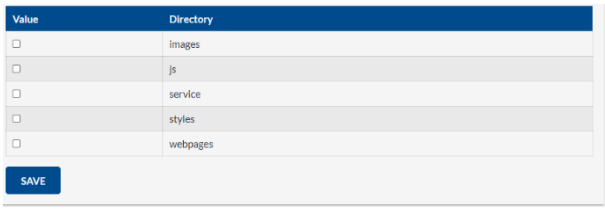
- **Enable /Disable** the access
- Define the **username** and the **password**

and configure the permission for a specific functionality in the control panel:



- **no permission**
- **read-only or read**
- **write access**

***C**ustom website folders*



8. Styles and CSS

In the custom website it is possible to manage the styles through the CSS files, like in any other website. Anyway, there are some HTML styles “restrictions” to keep in consideration while developing regarding the style of the specific HTML span used to read\write the variables:

- The items can be managed with a CSS file. In any case the rules of CSS will apply to the isavar and isamod items. This means that changing for example the color of the text for an isavar_* class will modify the text color of all the objects dynamically generated (input labels, buttons, checkboxes, etc.)
- The property id can be assigned only to those items that are included in the following classes:
  1) any kind of reading (isavar_*)
  2) isamod_booltog, isamod_boolwrt and isamod_dintwrt. Only these 3 writing objects can have an id working, because the graphical representation will be made by a single object

Clearly these restrictions are not considered while working in JavaScript, because the user will be able to look deeper into the objects to modify them.

9. REVISION

Revision 1.0.1 – Date: 2022/05/16

10. DISCLAIMER

This manual shall not be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior written consent from Emerson. Information in this document is believed to be accurate and reliable. However, Emerson does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. The customer(s) understand and agree that Emerson cannot confirm the manner in which the attached documentation is used. It is the customer's responsibility to review the advice provided herein and its applicability to the system. Emerson makes no representation about specific knowledge of the customer's system or the specific performance of the system. Emerson is not responsible for any damage to equipment or connected systems. The use of this document is at your own risk.

LUMITY™ EMERSON™

Dixell S.r.l. - Z.I. Via dell'Industria, 27 - 32016 Alpago (BL) ITALY
Tel. +39.0437.9833 r.a. - Fax +39.0437.989313 - EmersonClimate.com/Dixell - dixell@emerson.com

- Confidential. It is forbidden any use or reproduction without authorization of Dixell Srl -